

Jadavpur University

Department of Information Technology



SOFTWARE LAB – II · LABORATORY REPORT

Design and Implementation of a Project Management System

Submitted by

Name Sayanjib Sur

Roll No. 002511002004

Course ME-IT, 1st Year (2nd Semester)

Subject

Paper Software Lab – II

Code PG/ITE/S/121

Session 2025 – 2026

1. Problem Statement

Design a system, using object-oriented principles, for a company that handles multiple clients (projects) and assigns a specific set of employees to manage particular projects. Implement the whole system and generate a UML representation of it. The system must provide the following features:

1. assignment of employees to projects, recording the employee, the designation and the skill;
2. the project requirement, raised by a general department;
3. a project development tracker covering the life cycle of requirement and feasibility, testing, deployment and maintenance;
4. project billing; and
5. client feedback at each stage.

`Project` is to be taken as the super class, and further sub-classes are to be created as required.

2. Objective

The objectives of this assignment are:

- to identify the classes of the given problem domain along with their attributes, methods and mutual relationships;
- to represent the design as a UML class diagram;
- to implement the design in Java, using `Project` as an abstract super class;
- to validate the implementation with a set of test cases; and
- to build a graphical front end that operates on the same model.

3. Theory

The design applies the four standard relationships of a UML class diagram. *Inheritance* is used where one class is a specialised form of another. It allows the abstract operation `complexityFactor()` declared in `Project` to be redefined by each concrete project type, so that the billing module obtains the correct multiplier without knowing the actual type of the object it holds. This is *polymorphism*.

Aggregation is a whole-part relationship in which the part can exist on its own. An employee continues to exist if the company record is removed, so the link from `Company` to `Employee` is an aggregation. *Composition* is the stronger form, in which the part cannot exist without the whole. A `Billing` record has no meaning apart from the project it belongs to, so that link is a composition.

An *association class* is used when the link between two classes carries information of its own. The statement "which employee handles which project" is not merely a link, because it also carries the role the employee plays and the date the allocation was made. It is therefore modelled as the class `Assignment` rather than a plain association.

4. System Design

4.1 Identification of classes

The company is taken as the root of the system. It holds the clients, the employees and the projects, and it performs the allocation of an employee to a project. Each of the five features listed in the problem statement is given a class of its own, so that no single class carries unrelated responsibilities. The classes identified are listed in Table 1.

Class	Attributes (principal)	Responsibility
Company	name	Root class. Holds clients, employees and projects; allocates employees to projects.
Client	clientId, name, contactEmail	A customer of the company. Owns one or more projects.
Employee	employeeId, name, designation	A member of staff holding a designation and a set of skills.
Skill	name, level	A named competency held at a stated proficiency level.
Project (abstract)	projectId, name, status	Super class. Holds the requirement, tracker, billing, assignments and feedback of a project.
WebApplicationProject MobileAppProject DataAnalyticsProject	–	Concrete project types. Each supplies its own complexity factor.
Assignment	roleOnProject, assignedOn	Association class joining an employee to a project.
ProjectRequirement	department, summary, feasible	The requirement of a project, owned by a general department.
DevelopmentTracker	currentPhase, phaseCompletion	Tracks the current life-cycle phase and the completion of each phase.
Billing	baseCost, loggedHours, taxRate	Computes the invoice for a project.
ClientFeedback	phase, rating, comments	A rating and comment attached to one life-cycle phase.

Table 1: Classes identified in the problem domain.

Four enumerations support these classes: `Designation`, `LifeCyclePhase`, `ProjectStatus` and `SkillLevel`. Keeping the life-cycle phases in an enumeration fixes their order, which the tracker relies upon when it advances from one phase to the next.

4.2 Relationships between classes

- **Inheritance.** `WebApplicationProject`, `MobileAppProject` and `DataAnalyticsProject` extend the abstract class `Project`.

- **Aggregation.** Company aggregates Client, Employee and Project, each with multiplicity 1 to many.
- **Composition.** Each Project is composed of one ProjectRequirement, one DevelopmentTracker, one Billing and many ClientFeedback objects.
- **Association class.** Assignment joins Employee and Project with multiplicity many to many.
- **Dependency.** Classes depend on the enumerations that type their attributes.

4.3 UML class diagram

The complete class diagram is given in Figure 1 on the following page. As the diagram is wide, it has been printed sideways; the page should be turned clockwise to read it. It was prepared in PlantUML, and the source file `uml_class_diagram.puml` is included in the repository.

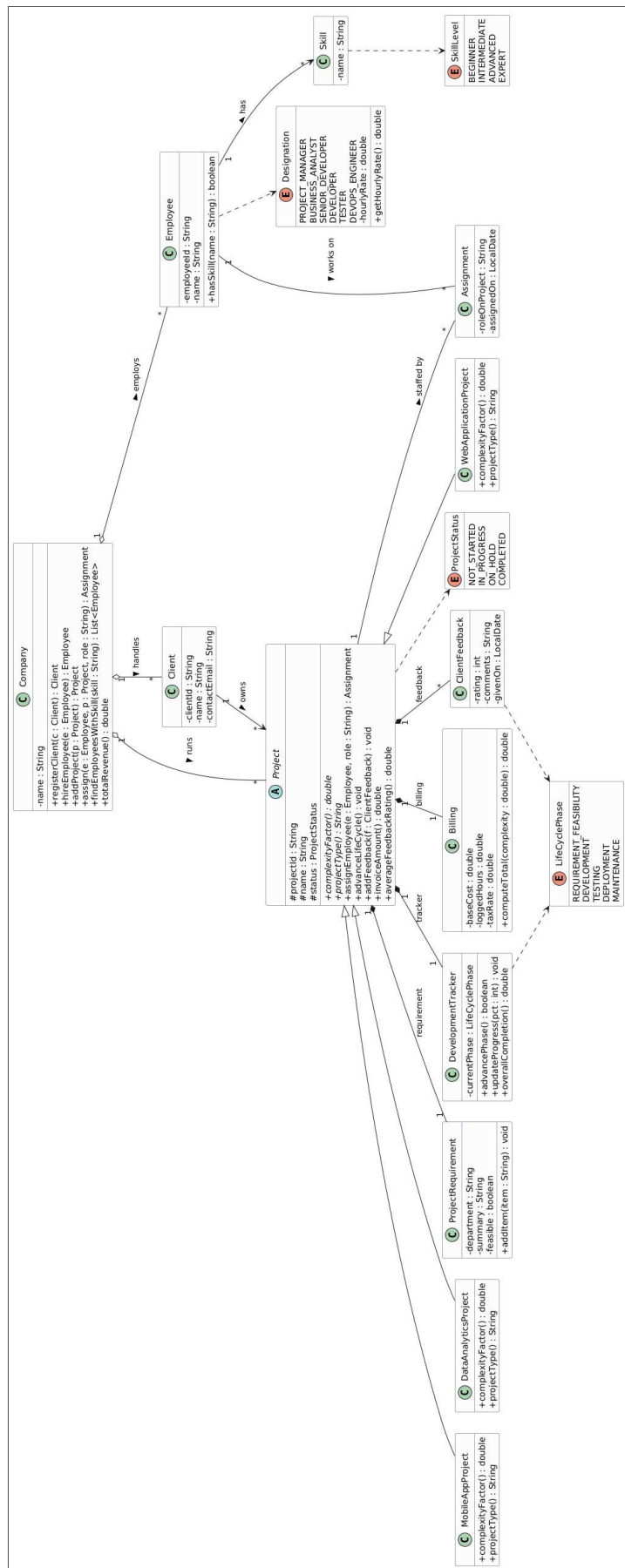


Figure 1: UML class diagram of the project management system.

5. Implementation

The system is implemented in Java (JDK 11) and uses no external libraries. The source is divided into four packages. `com.pms.enums` holds the four enumerations, `com.pms.model` holds the domain classes, `com.pms.app` holds the demonstration driver and the test suite, and `com.pms.ui` holds the graphical front end described in Section 7.

`Project` is declared abstract. It owns the requirement, the tracker, the billing record, the list of assignments and the feedback log, and declares the abstract operations `complexityFactor()` and `projectType()` that every concrete project type must supply. The method `invoiceAmount()` calls `complexityFactor()` without knowing which subclass is executing, and it is here that the polymorphism of the design is exercised. The three concrete types differ only in the factor they return: 1.2 for a web application, 1.4 for a mobile application and 1.6 for a data-analytics project.

Three points of the implementation are worth recording. First, both ends of the employee-to-project association are kept consistent: the constructor of `Assignment` registers the new link on the employee, while `Project.assignEmployee()` adds it to the project, so the model cannot reach a state in which one side knows of a link that the other does not. Second, invalid data is rejected at construction rather than stored, so a client-feedback rating outside the range one to five raises an exception. Third, because `LifeCyclePhase` is an enumeration, the tracker obtains the next phase from the ordinal of the current one, which guarantees that the phases of the life cycle cannot be visited out of order.

6. Testing

A test suite was written that requires no external framework, so that the program can be compiled and run with `javac` and `java` alone. It first builds a company of two clients, five employees and two projects and prints a report of that scenario, and then executes fifteen test cases. The cases and their results are given in Table 2. All fifteen cases pass.

No.	Test case	Expected	Observed	Result
1	<code>hasSkill()</code> returns true for a skill the employee holds	true	true	Pass
2	<code>hasSkill()</code> returns false for a skill not held	false	false	Pass
3	Assignment is recorded on the project	assignment present	assignment present	Pass
4	Assignment is recorded on the employee	assignment present	assignment present	Pass
5	A newly created project has status NOT_STARTED	NOT_STARTED	NOT_STARTED	Pass
6	Status becomes IN_PROGRESS after the first assignment	IN_PROGRESS	IN_PROGRESS	Pass
7	Tracker starts at phase REQUIREMENT_FEASIBILITY	REQUIREMENT_FEASIBILITY	REQUIREMENT_FEASIBILITY	Pass

No.	Test case	Expected	Observed	Result
8	advancePhase() moves the tracker to DEVELOPMENT	DEVELOPMENT	DEVELOPMENT	Pass
9	Status becomes COMPLETED after the full life cycle	COMPLETED	COMPLETED	Pass
10	A data-analytics project bills more than a web project	bill(1.6) > bill(1.2)	bill(1.6) > bill(1.2)	Pass
11	Web project invoice equals 1500 × 1.2	1800.00	1800.00	Pass
12	Feedback rating of 9 is rejected	IllegalArgumentException	IllegalArgumentException	Pass
13	Average of ratings 4 and 2	3.0	3.0	Pass
14	findEmployeesWithSkill("Go") matches one employee	1	1	Pass
15	totalRevenue() over two projects (1000×1.2 + 1000×1.6)	2800.00	2800.00	Pass

Table 2: Test cases and their results.

The output produced by the program is shown in Figure 2. The upper portion is the report of the demonstration scenario and the lower portion is the result of the test suite.

```
Terminal - ProjectManagementDemo

=====
PROJECT MANAGEMENT SYSTEM - DEMO SCENARIO
=====
Company: Acme Software Pvt. Ltd.
Total clients : 2
Total employees: 5
Total projects : 2

=====
WebApplicationProject "RetailKart Storefront" [P-01] client=RetailKart status=COMPLETED type=Web Application
Requirement : Requirement[dept=E-Commerce, feasible=true]: Online storefront with cart & payments [Product catalogue, Shopping cart, Payment gateway]
Life cycle : Tracker[current=Maintenance] Requirement & Feasibility=100% Development=100% Testing=100% Deployment=100% Maintenance=0%
Completion : 80%
Assignments (which employee handles this project):
- Alice handles "RetailKart Storefront" as Project Manager (since 2026-07-08)
- Bob handles "RetailKart Storefront" as Lead Developer (since 2026-07-08)
- Dan handles "RetailKart Storefront" as QA Engineer (since 2026-07-08)
Client feedback at each stage:
- [Requirement & Feasibility] 5/5 - "Scope looks great" (2026-07-08)
- [Development] 4/5 - "Good progress on cart" (2026-07-08)
- [Testing] 4/5 - "Few bugs, fixed quickly" (2026-07-08)
- [Deployment] 5/5 - "Smooth go-live" (2026-07-08)
Avg feedback: 4.50 / 5
Billing[base=5000.00, hours=100.0, rate=55.00, tax=10%, paid=false]
Invoice (incl. tax, complexity x1.2): 21098.40

=====
MobileAppProject "MediCare Patient App" [P-02] client=MediCare Labs status=IN_PROGRESS type=Mobile Application
Requirement : Requirement[dept=Healthcare, feasible=true]: Patient appointment & records app [Appointment booking, Lab report viewer]
Life cycle : Tracker[current=Requirement & Feasibility] Requirement & Feasibility=0% Development=0% Testing=0% Deployment=0% Maintenance=0%
Completion : 0%
Assignments (which employee handles this project):
- Carol handles "MediCare Patient App" as Android Developer (since 2026-07-08)
- Eve handles "MediCare Patient App" as DevOps (since 2026-07-08)
Client feedback at each stage:
- [Requirement & Feasibility] 4/5 - "Clear requirements" (2026-07-08)
Avg feedback: 4.00 / 5
Billing[base=7000.00, hours=90.0, rate=60.00, tax=10%, paid=false]
Invoice (incl. tax, complexity x1.4): 20484.80

=====
TOTAL COMPANY REVENUE: 41583.20
=====

=====
TEST CASES
=====
[PASS] Employee.hasSkill finds an existing skill
[PASS] Employee.hasSkill rejects a missing skill
[PASS] Assignment appears on the project
[PASS] Assignment appears on the employee
[PASS] Project starts NOT_STARTED
[PASS] Project becomes IN_PROGRESS after first assignment
[PASS] Tracker starts at REQUIREMENT_FEASIBILITY
[PASS] Tracker advances to DEVELOPMENT
[PASS] Project is COMPLETED after full life cycle
[PASS] Higher complexity yields higher bill
[PASS] Web invoice equals 1500 * 1.2 = 1800
[PASS] Feedback rating outside 1.5 is rejected
[PASS] Average feedback is (4+2)/2 = 3.0
[PASS] findEmployeesWithSkill returns only matching employees
[PASS] totalRevenue aggregates all projects

=====
RESULT: 15 passed, 0 failed, 15 total
=====
```

Figure 2: Output of ProjectManagementDemo.

7. Graphical User Interface

A desktop front end was written over the same model classes using Swing, which is part of the standard library, so that no external dependency is introduced. The window is divided into a navigation panel on the left and one of four views on the right. The dashboard summarises the portfolio, the project view shows the life cycle, team, requirement, feedback and billing of each project, and the remaining two views list the employees and the clients. The interface can be displayed in a light or a dark appearance.

The project view is not merely a display. Pressing the button marked *Advance life cycle* calls `Project.advanceLifeCycle()` on the underlying object, and the progress bars, the status and the invoice are redrawn from the new state of the model.

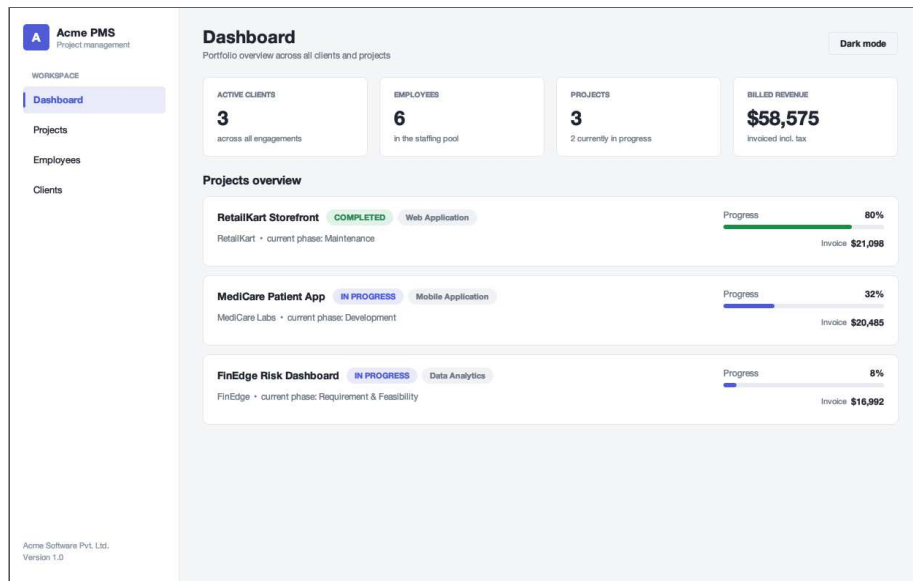


Figure 3: Dashboard view.

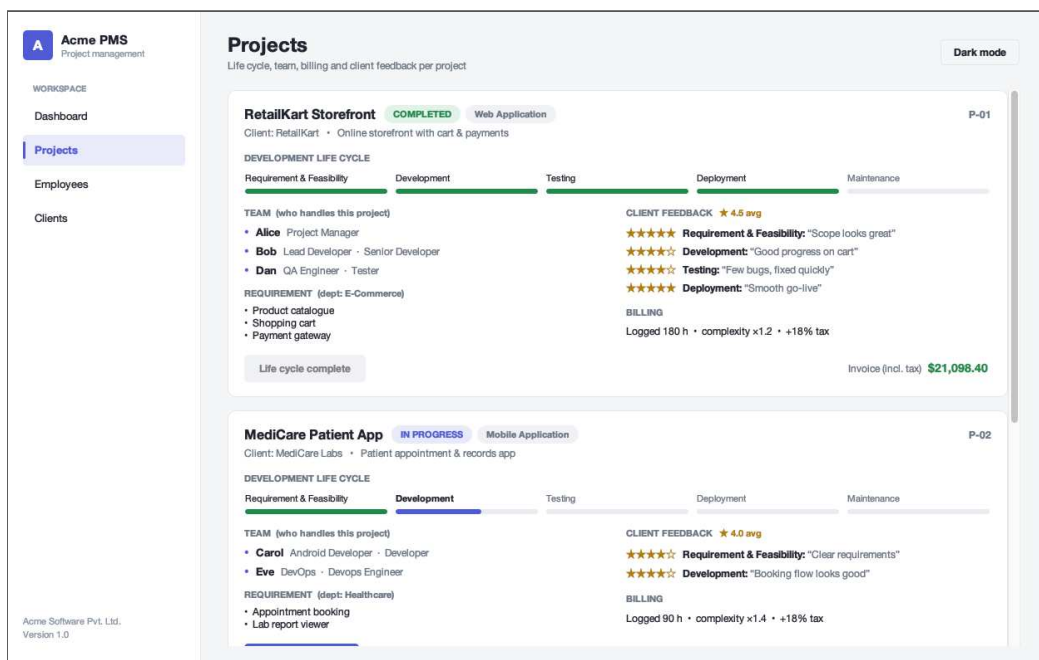


Figure 4: Project view, showing the life-cycle tracker, the assigned team, the requirement, the client feedback and the billing of each project.

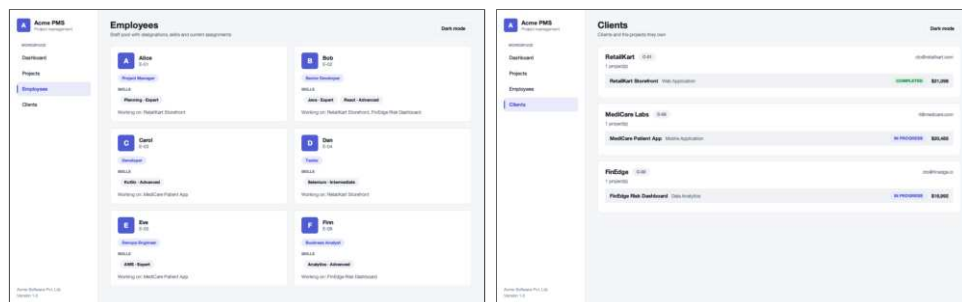


Figure 5: Employee view (left) and client view (right).

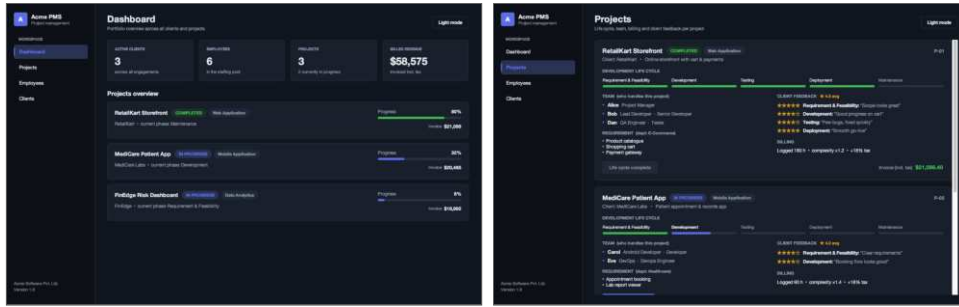


Figure 6: The same two views in the dark appearance.

8. Results and Discussion

All fifteen test cases pass, and the demonstration scenario produces the expected invoice of 21,098.40 for the web project and 20,484.80 for the mobile project. The difference between the two follows from the complexity factors 1.2 and 1.4 respectively, which confirms that the abstract method is being resolved to the correct subclass at run time.

Taking `Project` as an abstract super class keeps the cost of change low. A new type of project requires one small subclass and no change anywhere else, because every other class refers to the super class only. Similarly, a new life-cycle phase or a new designation requires only a new constant in the corresponding enumeration.

The chief limitation of the present implementation is that all data is held in memory and is lost when the program terminates. The billing model is also a simplification, in that it applies a single blended hourly rate to a project rather than charging each employee at the rate of their own designation, although the rate is already stored against the designation and could be used directly.

9. Source Code and Execution

The complete source code, the UML source file, the build scripts and the screenshots reproduced in this report are available in the following public repository:

<https://github.com/Codewithsayanjib/project-management-system-uml>

The project has no external dependencies, so a Java Development Kit (version 11 or later) is the only requirement. To compile and run it:

- Clone the repository and change into it: `git clone <repository URL> && cd project-management-system-uml`
- Compile all the sources into the out directory: `find src -name "*.java" > sources.txt && javac -d out @sources.txt`
- Run the demonstration and the test suite: `java -cp out com.pms.app.ProjectManagementDemo`
- Build and launch the graphical front end: `./build_app.sh` followed by `java -jar dist/AcmePMS.jar`